Конспект - рекуррентные нейронные сети для начинающих

Содержание

1. Введение: Зачем нужны RNN?
2. Простые RNN: Основы
3. LSTM: Долгая память
4. GRU: Упрощенная версия
5. Сравнение всех архитектур
6. Практические примеры

Введение: Зачем нужны RNN?

Проблема обычных нейронных сетей

Представьте, что вы читаете предложение: **“Кот сидел на…”**. Чтобы понять, что будет дальше, вам нужно помнить предыдущие слова. Обычная нейронная сеть (полносвязная) обрабатывает каждое слово независимо и **не помнит** предыдущие слова.

Аналогия: Это как если бы вы читали книгу, но после каждого слова у вас стиралась память о предыдущих словах. Понять смысл было бы невозможно!

Что такое последовательные данные?

Последовательные данные - это данные, где **порядок имеет значение**:

- **Текст:** “Я люблю кота” ≠ “Кота я люблю” ≠ “Люблю я кота”
- **Временные ряды:** Цена акций сегодня зависит от вчерашней цены
- **Речь:** Звуки в слове должны идти в правильном порядке
- **ДНК:** Последовательность генов определяет свойства организма

Решение: Рекуррентные нейронные сети

RNN (Recurrent Neural Networks) - это нейронные сети с **памятью**. Они могут “помнить” информацию из предыдущих шагов и использовать её для обработки текущего входа.

Простые RNN: Основы

Как работает память в RNN?

Представьте RNN как **умного секретаря**, который: 1. Получает новую информацию (входной сигнал) 2. Вспоминает, что было раньше (предыдущее состояние) 3. Объединяет новое и старое 4. Выдает результат и запоминает его для следующего раза

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

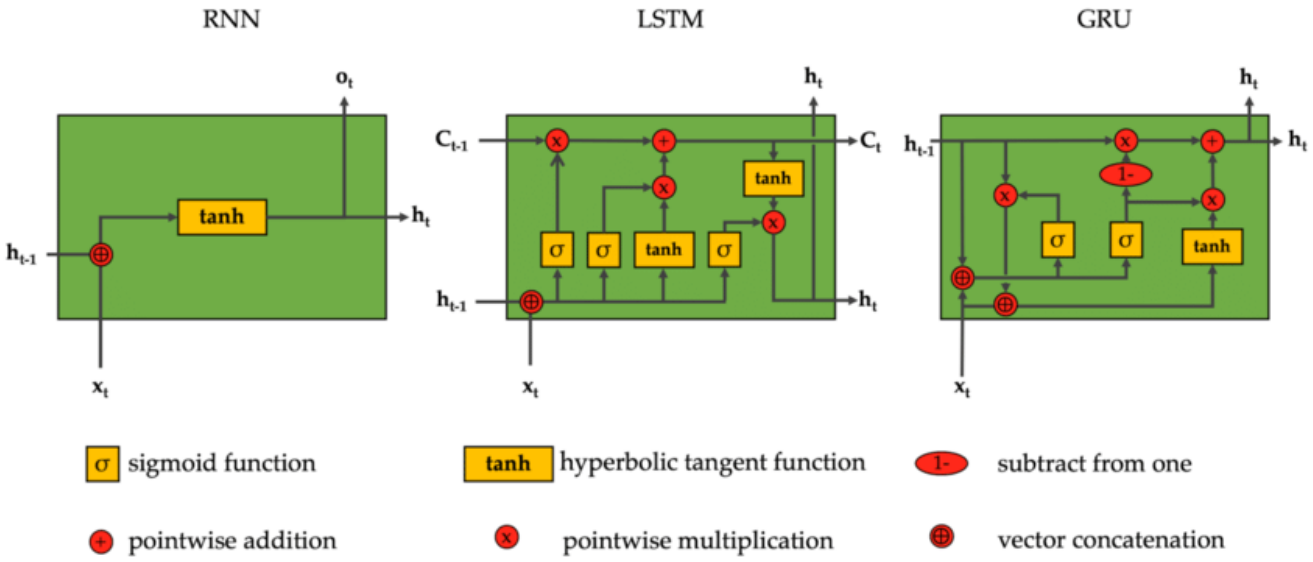

Рис. 1. Сравнение архитектур: RNN запоминает информацию между шагами

Математика RNN: Пошагово

Шаг 1: Что у нас есть?

- x_t - **входной сигнал** в момент времени t (например, слово в предложении)
- h_{t-1} - **память** о предыдущем шаге (что мы помним)
- h_t - **новая память** (что мы запомним сейчас)
- y_t - **выход** (наш ответ)

Шаг 2: Как вычисляется новая память?

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

Разберем по частям:

1. $W_{xh} \cdot x_t$ - "обрабатываем новую информацию"
   - W_{xh} - матрица весов (как важна новая информация?)
   - Умножение означает: "насколько сильно новая информация влияет на память?"
2. $W_{hh} \cdot h_{t-1}$ - "вспоминаем старую информацию"
   - W_{hh} - матрица весов (как важна старая память?)
   - Умножение означает: "насколько сильно старая память влияет на новую?"
3. $+b_h$ - "добавляем смещение"
   - Это как "базовый уровень активности" нейрона
4. $\tanh(\ldots)$ - "нормализуем результат"
   - Функция tanh сжимает результат в диапазон от -1 до 1
   - Это предотвращает "взрыв" значений

Шаг 3: Как получается выход?

$$y_t = W_{hy} \cdot h_t + b_y$$

Простыми словами: Берем нашу новую память h_t, умножаем на веса W_{hy} и получаем ответ.

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

Пример работы RNN

Допустим, мы обрабатываем предложение: **“Кот сидел на”**

Шаг 1: Входное слово = “Кот” - x_1 = [вектор для слова “Кот”] - h_0 = [0, 0, 0] (изначально память пуста) - h_1 = $\tanh(W_{hh} \cdot [0, 0, 0] + W_{xh} \cdot x_1 + b_h)$ - Результат: h_1 = [0.2, -0.1, 0.5] (запомнили про кота)

Шаг 2: Входное слово = “сидел” - x_2 = [вектор для слова “сидел”] - h_1 = [0.2, -0.1, 0.5] (помним про кота) - h_2 = $\tanh(W_{hh} \cdot [0.2, -0.1, 0.5] + W_{xh} \cdot x_2 + b_h)$ - Результат: h_2 = [0.1, 0.3, -0.2] (помним: кот + сидел)

Шаг 3: Входное слово = “на” - И так далее…

Проблемы простых RNN

1. Проблема исчезающих градиентов

Что происходит? При обучении RNN информация из далекого прошлого “забывается”.

Почему? При обратном распространении ошибки градиенты умножаются много раз на одни и те же веса. Если веса меньше 1, то произведение стремится к нулю.

Аналогия: Это как игра “испорченный телефон” - чем дальше передается сообщение, тем больше оно искажается.

Математически:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=1}^{k} \frac{\partial h_{t-i+1}}{\partial h_{t-i}}$$

Если каждая частная производная меньше 1, то произведение быстро уменьшается.

2. Проблема взрывающихся градиентов

Что происходит? Градиенты становятся очень большими и “ломают” обучение.

Почему? Если веса больше 1, то произведение растет экспоненциально.

Решение: Обрезка градиентов (gradient clipping) - если градиент больше порога, обрезаем его.

LSTM: Долгая память

Зачем нужны LSTM?

LSTM (Long Short-Term Memory) решают проблему исчезающих градиентов с помощью **умных ворот**, которые контролируют поток информации.

Аналогия: LSTM - это как умный архивариус, который решает: - Что забыть из старой информации - Что запомнить из новой информации
- Что показать наружу

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

Архитектура LSTM: Четыре компонента

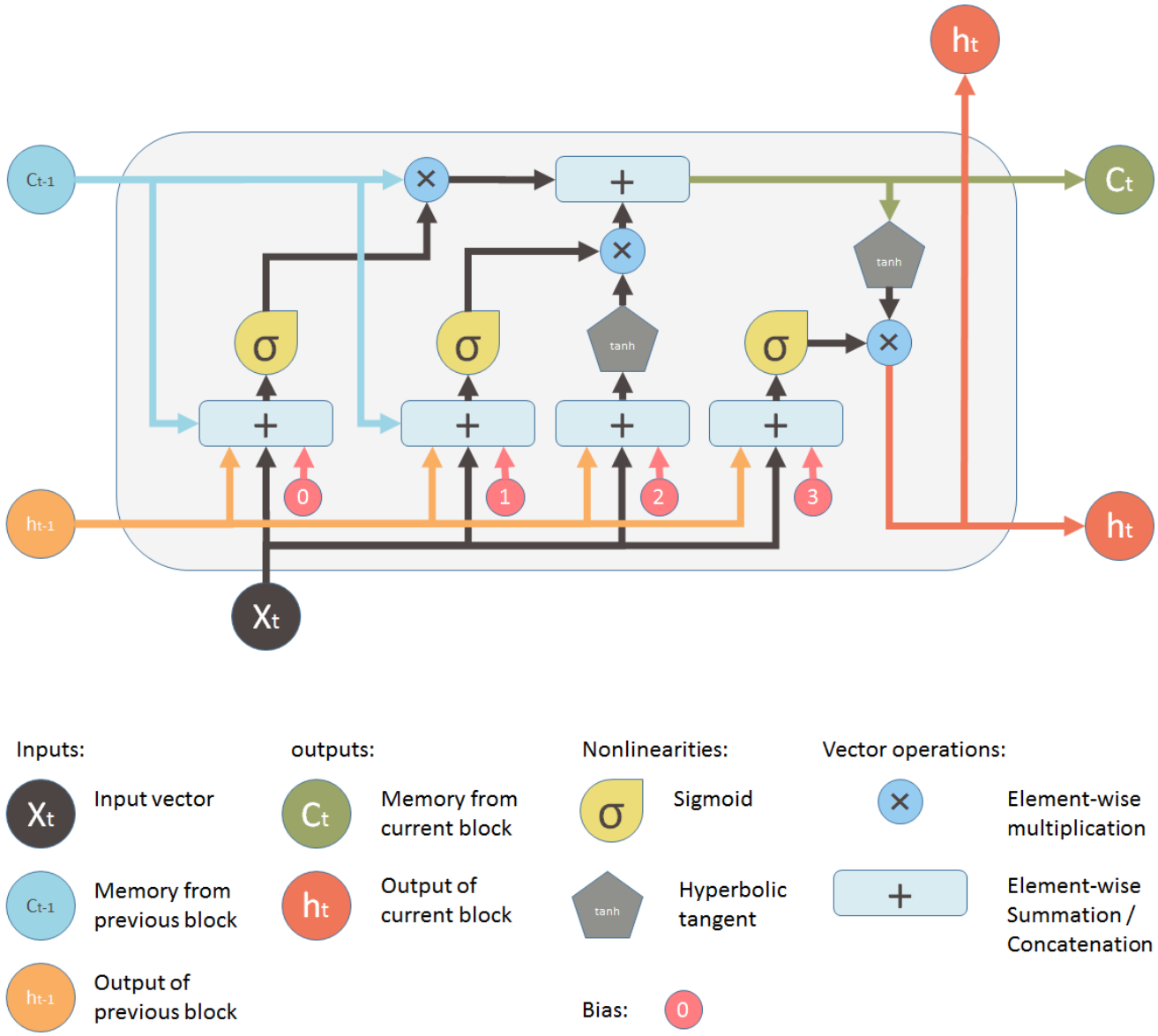

Рис. 2. Детальная схема ячейки LSTM с воротами

1. Cell State (Состояние ячейки) - C_t

Что это? Главная “память” LSTM - информация течет через неё почти без изменений.

Аналогия: Это как конвейерная лента на заводе - информация движется по ней, а рабочие (ворота) решают, что добавить или убрать.

2. Forget Gate (Ворота забывания) - f_t

Задача: Решить, какую информацию из старой памяти забыть.

Формула:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Разбор по шагам:

1. $[h_{t-1}, x_t]$ - объединяем старую память и новый вход
   - Если $h_{t-1} = [0.2, 0.1]$ и $x_t = [0.5, -0.3]$
   - То $[h_{t-1}, x_t] = [0.2, 0.1, 0.5, -0.3]$

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

2. $W_f \cdot [h_{t-1}, x_t]$ - применяем веса
   - Веса “решают”, какие комбинации входа и памяти важны
3. $\sigma(\ldots)$ - сигмоидная функция
   - Выдает значения от 0 до 1
   - 0 = “забыть полностью”, 1 = “помнить полностью”

Пример: Если мы читаем “Кот был черный. Собака…”, то при слове “Собака” ворота забывания могут решить забыть информацию о коте.

3. Input Gate (Входные ворота) - i_t

Задача: Решить, какую новую информацию сохранить в памяти.

Состоит из двух частей:

Часть A: Что обновлять?

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Часть B: Какие новые значения?

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Объяснение: - i_t решает, **какие части** памяти обновить (значения от 0 до 1) - $\tilde{C}_t$ содержит **новые значения** для обновления (от -1 до 1)

4. Обновление состояния ячейки

Формула:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Разбор: - $f_t \odot C_{t-1}$ - “забываем” часть старой памяти - $i_t \odot \tilde{C}_t$ - “добавляем” новую информацию - $\odot$ - поэлементное умножение

Пример с числами: - Старая память: $C_{t-1} = [0.8, -0.2, 0.5]$ - Ворота забывания: $f_t = [0.1, 0.9, 0.0]$ (забыть первое и третье) - Входные ворота: $i_t = [0.8, 0.1, 0.9]$ (обновить первое и третье) - Новые значения: $\tilde{C}_t = [0.6, 0.3, -0.4]$

Результат:

$$C_t = [0.1 \times 0.8, 0.9 \times (-0.2), 0.0 \times 0.5] + [0.8 \times 0.6, 0.1 \times 0.3, 0.9 \times (-0.4)]$$

$$C_t = [0.08, -0.18, 0.0] + [0.48, 0.03, -0.36] = [0.56, -0.15, -0.36]$$

5. Output Gate (Выходные ворота) - o_t

Задача: Решить, какую часть памяти показать наружу.

Формулы:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Объяснение: - o_t решает, **что показать** из памяти - $\tanh(C_t)$ нормализует память в диапазон [-1, 1] - h_t - итоговое скрытое состояние (то, что “видят” другие слои)

Почему LSTM работают лучше?

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

1. Прямой путь для градиентов

В состоянии ячейки C_t информация течет почти без изменений:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Градиент по C_{t-1} равен просто f_t, без сложных умножений на веса.

2. Селективная память

Ворота позволяют: - **Забывать** ненужную информацию (forget gate) - **Запоминать** важную информацию (input gate) - **Контролировать** выход (output gate)

GRU: Упрощенная версия

Идея GRU

GRU (Gated Recurrent Unit) - это “упрощенная LSTM” с двумя воротами вместо трех.

Основные отличия: - Нет отдельного состояния ячейки (используется только h_t) - Объединены forget и input gates в один update gate - Меньше параметров = быстрее обучение

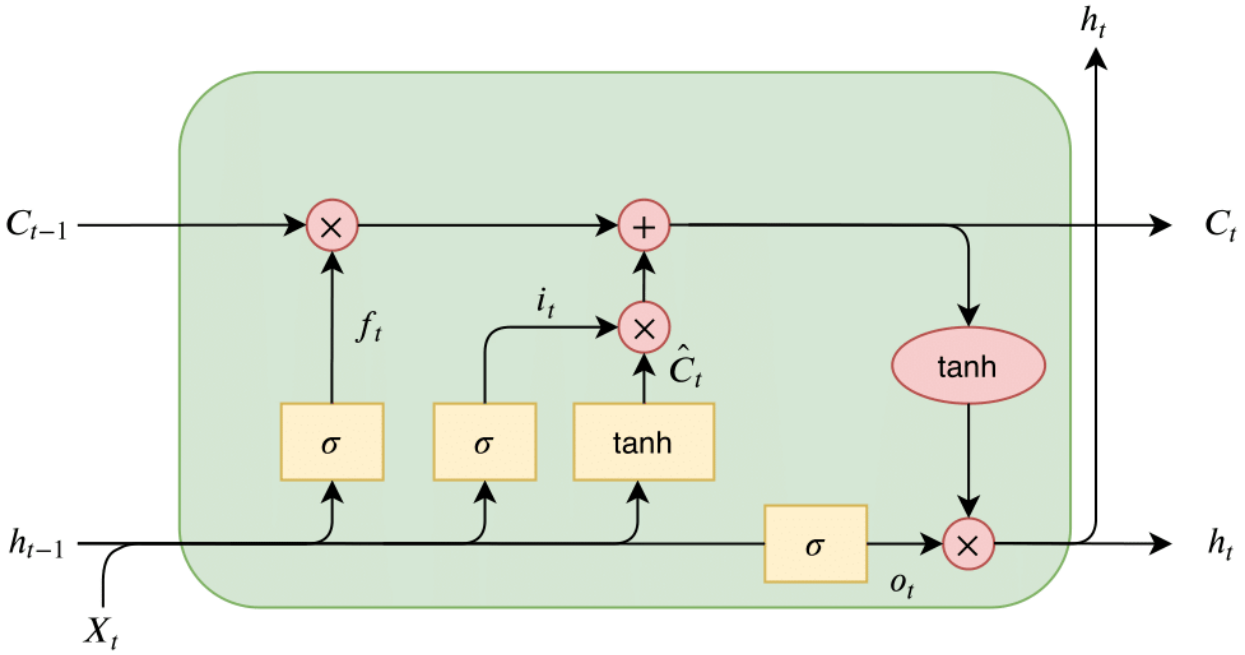

Рис. 3. Схема ячейки GRU с двумя воротами

Компоненты GRU

1. Update Gate (Ворота обновления) - z_t

Задача: Решить, сколько старой информации сохранить и сколько новой добавить.

Формула:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

Интуиция: - $z_t = 0$ означает “забыть все старое, использовать только новое” - $z_t = 1$ означает “сохранить все старое,

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

игнорировать новое” - $z_t = 0.3$ означает “70% нового, 30% старого”

2. Reset Gate (Ворота сброса) - r_t

Задача: Решить, насколько предыдущая память важна для вычисления новой информации.

Формула:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Интуиция: - $r_t = 0$ означает “игнорировать предыдущую память при создании новой” - $r_t = 1$ означает “полностью использовать предыдущую память”

3. Candidate Hidden State (Кандидат нового состояния) - $\tilde{h}_t$

Задача: Создать “предложение” для нового состояния памяти.

Формула:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

Разбор: - $r_t \odot h_{t-1}$ - “отфильтрованная” старая память - $[r_t \odot h_{t-1}, x_t]$ - объединяем отфильтрованную память с новым входом - $\tanh(\ldots)$ - нормализуем в диапазон [-1, 1]

4. Final Hidden State (Итоговое состояние) - h_t

Формула:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Разбор: - $(1-z_t)\odot h_{t-1}$ - “доля старой памяти” - $z_t \odot \tilde{h}_t$ - “доля новой информации” - Сумма дает итоговое состояние

Пример с числами: - Старое состояние: $h_{t-1} = [0.5, -0.2]$ - Update gate: $z_t = [0.3, 0.7]$ - Кандидат: $\tilde{h}_t = [0.8, 0.1]$

Результат:

$$h_t = [0.7 \times 0.5, 0.3 \times (-0.2)] + [0.3 \times 0.8, 0.7 \times 0.1]$$

$$h_t = [0.35, -0.06] + [0.24, 0.07] = [0.59, 0.01]$$

Пошаговый пример работы GRU

Обрабатываем предложение: **“Я изучаю нейросети”**

Инициализация: - $h_0 = [0, 0]$ (пустая память)

Шаг 1: Слово “Я” - x_1 = вектор для “Я” - $z_1 = \sigma(W_z \cdot [h_0, x_1]) = [0.6, 0.4]$ (больше обновляем первый элемент) - $r_1 = \sigma(W_r \cdot [h_0, x_1]) = [0.2, 0.8]$ (мало используем старую память) - $\tilde{h}_1 = \tanh(W_h \cdot [r_1 \odot h_0, x_1]) = [0.3, -0.1]$ - $h_1 = (1 - z_1) \odot h_0 + z_1 \odot \tilde{h}_1 = [0.18, -0.04]$

Шаг 2: Слово “изучаю” - x_2 = вектор для “изучаю” - $z_2 = [0.4, 0.7]$ (больше обновляем второй элемент) - $r_2 = [0.9, 0.5]$ (хорошо используем память о “Я”) - $\tilde{h}_2 = [0.2, 0.6]$ - $h_2 = [0.6 \times 0.18, 0.3 \times (-0.04)] + [0.4 \times 0.2, 0.7 \times 0.6] = [0.188, 0.408]$

И так далее…

Сравнение всех архитектур

Таблица сравнения

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

Характеристика	Simple RNN	LSTM	GRU
Количество ворот	0	3 (forget, input, output)	2 (update, reset)
Отдельная память	Нет	Да (Cell State)	Нет
Параметры	Мало	Много	Средне
Скорость обучения	Быстро	Медленно	Средне
Долгосрочная память	Плохо	Отлично	Хорошо
Сложность реализации	Простая	Сложная	Средняя

Когда что использовать?

Simple RNN

Используйте когда: - Короткие последовательности (< 10 элементов) - Ограниченные вычислительные ресурсы - Простые задачи

Примеры: Классификация коротких текстов, простое предсказание временных рядов

LSTM

Используйте когда: - Длинные последовательности (> 50 элементов) - Важны долгосрочные зависимости - Есть достаточно данных для обучения

Примеры: Машинный перевод, анализ длинных документов, сложные временные ряды

GRU

Используйте когда: - Средние последовательности (10-50 элементов) - Нужен баланс между качеством и скоростью - Ограниченные данные

Примеры: Анализ тональности, генерация текста, распознавание речи

Практические примеры

Пример 1: Предсказание следующего слова

Задача: По началу предложения предсказать следующее слово.

Вход: “Сегодня хорошая” **Ожидаемый выход:** “погода”

Как работает RNN: 1. Обрабатывает “Сегодня” → запоминает контекст времени 2. Обрабатывает “хорошая” → понимает, что речь о чем-то положительном 3. Предсказывает “погода” как наиболее вероятное продолжение

Пример 2: Анализ тональности

Задача: Определить, положительный или отрицательный отзыв.

Вход: “Фильм был скучный и длинный” **Выход:** Отрицательный (0.2 из 1.0)

Как работает LSTM: 1. “Фильм” → нейтральный контекст 2. “был” → связка, запоминается 3. “скучный” → отрицательная оценка, сохраняется в памяти 4. “и” → связка 5. “длинный” → дополнительная отрицательная оценка 6. Итоговое решение: отрицательный отзыв

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/

Пример 3: Прогнозирование временного ряда

Задача: Предсказать цену акций на завтра.

Вход: Цены за последние 30 дней **Выход:** Цена на завтра

Как работает GRU: 1. Анализирует тренд (растет/падает) 2. Учитывает сезонность (день недели, месяц) 3. Запоминает важные события (новости, отчеты) 4. Делает прогноз на основе всей накопленной информации

Заключение

Ключевые выводы

1. **RNN решают проблему последовательных данных**, где важен порядок элементов
2. **Simple RNN просты, но имеют проблемы с долгосрочной памятью**
3. **LSTM используют ворота для контроля информации** и хорошо работают с длинными последовательностями
4. **GRU - это упрощенная версия LSTM** с меньшим количеством параметров
5. **Выбор архитектуры зависит от задачи** и доступных ресурсов

Дальнейшее изучение

После освоения базовых RNN рекомендуется изучить: - **Bidirectional RNN** (двунаправленные сети) - **Attention механизмы** (механизмы внимания) - **Transformer архитектуры** (современная альтернатива RNN) - **Seq2Seq модели** (последовательность в последовательность)

Полезные ресурсы

1. **Understanding LSTM Networks** - отличная визуализация
2. **The Unreasonable Effectiveness of Recurrent Neural Networks** - практические примеры
3. **Deep Learning Book** - теоретические основы

© Dmitry Vlasov 2025. All rights reserved.
Licensed under CC BY-NC-ND 4.0 – https://creativecommons.org/licenses/by-nc-nd/4.0/